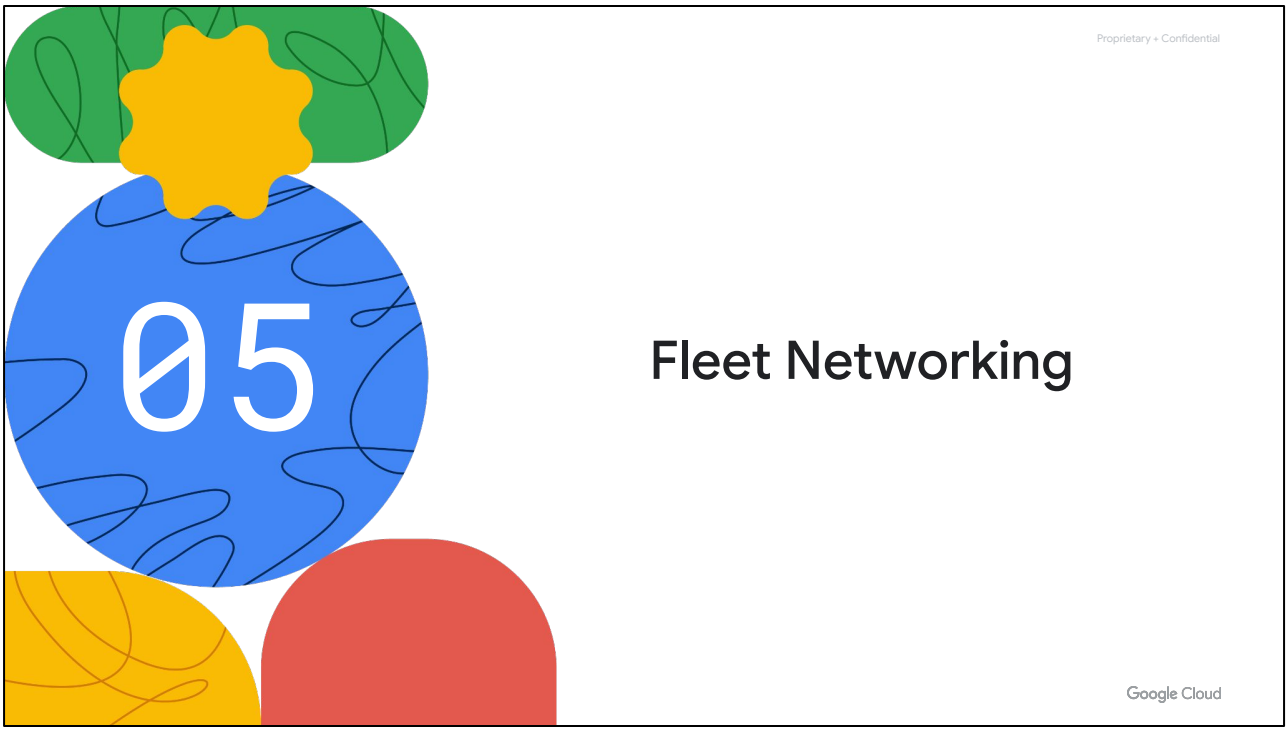


Say: Efficient and reliable communication between clusters is critical. In this section, we'll dive into the world of interconnected Kubernetes clusters and unlock the potential of seamless communication across your deployments.

Imagine a world where your applications span multiple regions, availability zones, and even different cloud providers, all while operating as a unified, powerful entity. That's the promise of Fleet Networking in Google Kubernetes Engine (GKE).

In today's dynamic cloud landscape, applications are rarely confined to a single Kubernetes cluster. Whether you're aiming for high availability, disaster recovery, or simply expanding your reach, managing multiple clusters is becoming the norm. But with this expansion comes the challenge of connectivity. How do you ensure your microservices, spread across different clusters, can communicate efficiently and securely?

This is where fleet networking steps in. By providing a unified network fabric for your GKE clusters, it eliminates the complexities of traditional networking.

Proprietary + Confidential

Fleet Networking

01 Fleet networking communications

02 Pod discovery in GKE

03 Multi-cluster Services

04 Configuring multi-cluster Services

05 Multi-cluster gateway

06 Configuring multi-cluster gateways

Google Cloud

Say: In this section of the course, titled “Fleet Networking” you’ll learn to:

- Explain how fleet networking works.
- Describe how Pods in a Kubernetes cluster communicate with each other.
- Enable multi-cluster Services.
- Configure multi-cluster Services.
- Detail the elements of fleet management.
- Outline the role of a multi-cluster gateway.
- Configure a multicluster gateway

Proprietary + Confidential

Fleet Networking

01 Fleet networking communications

02 Pod discovery in GKE

03 Multi-cluster Services

04 Configuring multi-cluster Services

05 Multi-cluster gateway

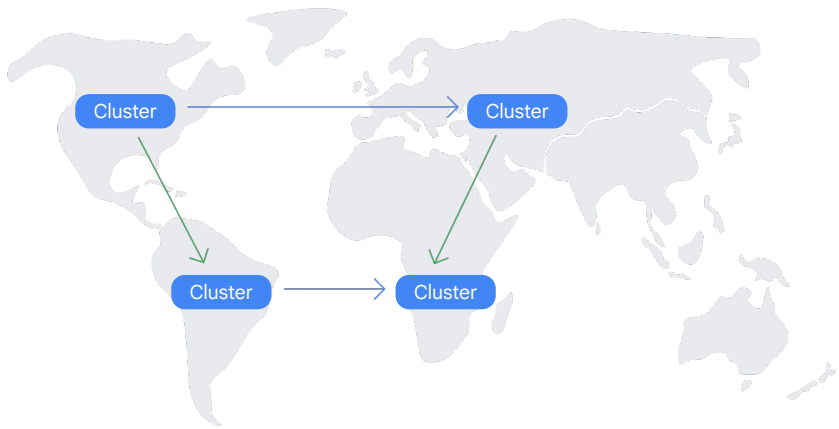
06 Configuring multi-cluster gateways

Google Cloud

Say: To begin, what is fleet networking in GKE, and how does it work?

Proprietary + Confidential

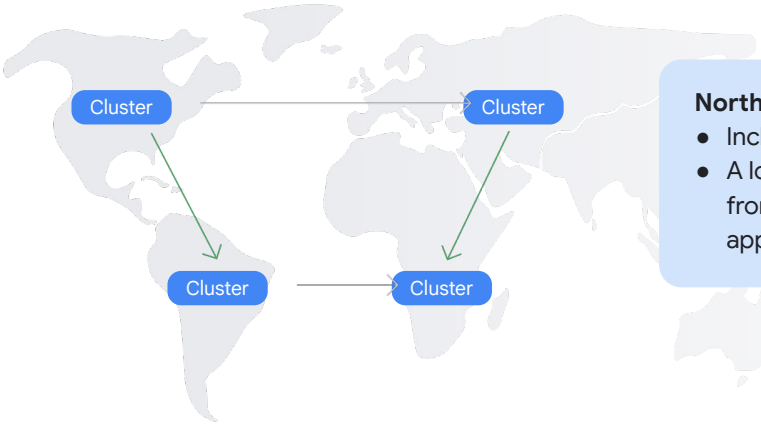
In GKE, clusters communicate directionally



Google Cloud

Say: In GKE, clusters communicate directionally: north-south and east-west.

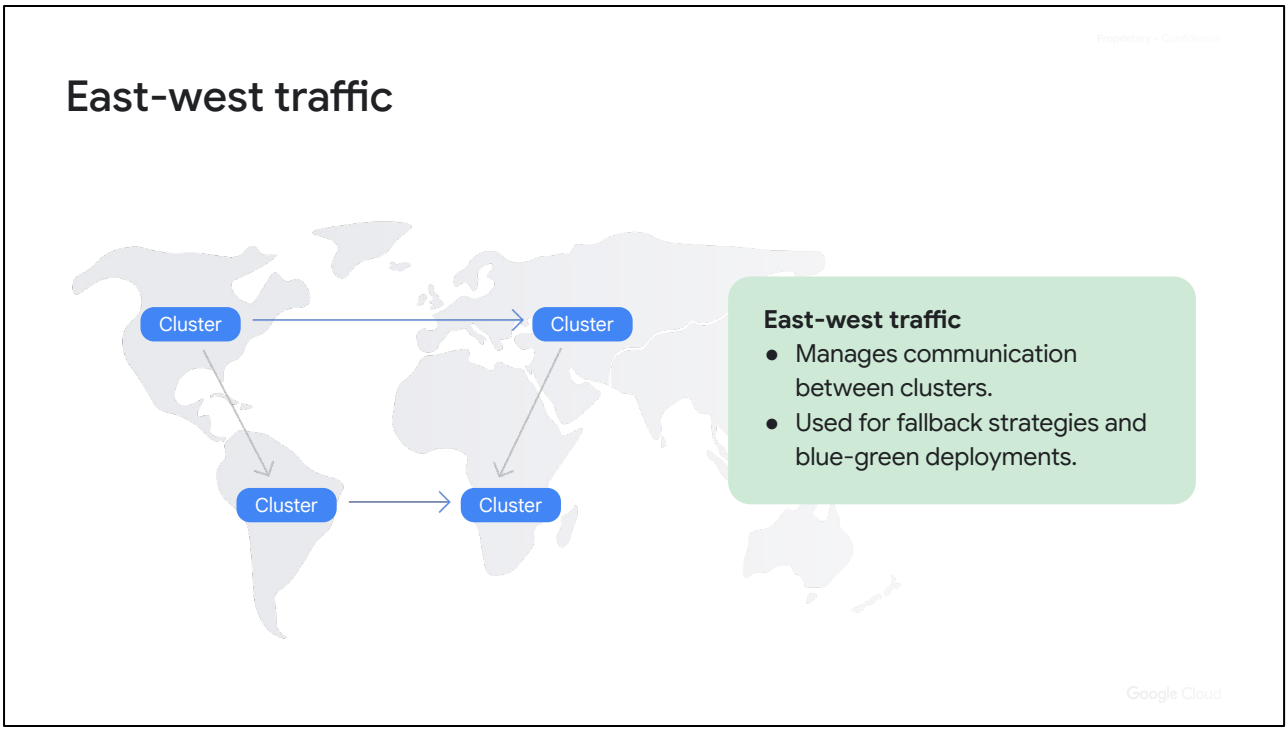
North-south traffic



North-south traffic

- Includes Ingress and egress.
- A load balancer receives traffic from the internet or internal applications.

Say: North-south traffic includes ingress and egress traffic, and communicates with clusters via a load balancer. The load balancer receives traffic from the internet or from internal applications.



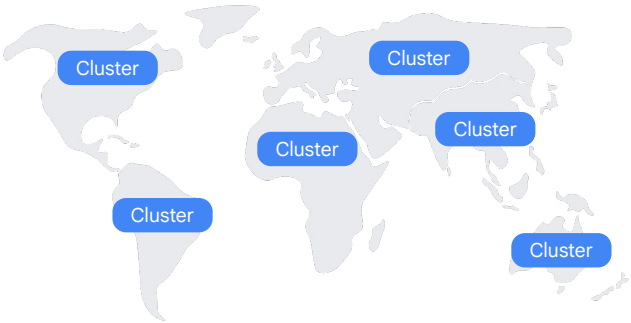
Say: Next let's explore east-west routing, which manages communication between clusters.

East-west traffic is essential if you have dependencies between services in different clusters. It is also used to implement fallback strategies and perform blue-green deployments.

Cloud Service Mesh is a key component for managing east-west traffic, providing service discovery, traffic management, and security features for services that communicate across clusters.

Proprietary + Confidential

Hosting clusters in multiple locations



- Proximity to customers
- Availability
- Compliance
- Data locality

Google Cloud

Say: Let's examine why companies often choose to host clusters in different locations.

One reason is **proximity to customers.** Companies close to customers can serve traffic with minimum latency.

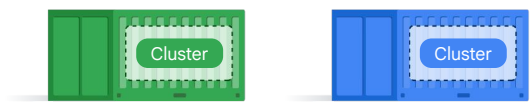
Another benefit is increased **availability.** Redundancy provides resilience and recovery if a region becomes unavailable.

Compliance is also important. Companies often choose to locate clusters in the same jurisdiction as their customers meet data residency requirements.

Many customers also benefit from **data locality**. Keeping compute resources close to data reduces costs and increases performance.

Proprietary + Confidential

Multiple clusters can help achieve isolation



- Environment and workload isolation
- Limit the impact of failure
- Upgrade flexibility
- Separate sensitive data from applications

Google Cloud

Say: Multiple clusters can also help achieve isolation.

For example, you can **use separate clusters for different environments**, like development and production workloads.

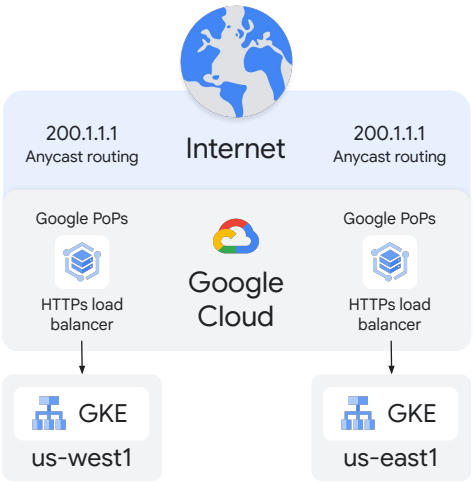
Workload categorization: Splitting workloads into different categories.

Isolation can also help **limit the impact of failure** and **provide upgrade flexibility** without reducing availability.

And with multiple clusters, you can separate sensitive data from applications, ensuring compliance and security.

Proprietary + Confidential

Multi-cluster networking



- **Anycast IP** can be deployed to multiple regions, routing traffic to the nearest cluster.
- **Global HTTP(S) load balancers** use Anycast IP addresses to receive and distribute traffic.
- Google's Premium Network optimizes routing.

Google Cloud

Say: In GKE, several technologies work together to enable multi-cluster networking.

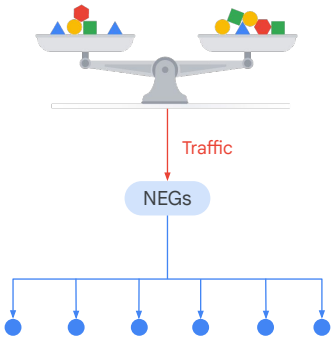
Anycast IP is an IP address that can be deployed to multiple regions, routing traffic to the nearest cluster.

Global HTTPs load balancers use anycast IPs and network endpoint groups (NEGs) to distribute traffic efficiently.

GKE uses Google's Premium Network for optimized routing. Traffic from outside Google's network is quickly handed off to the nearest Google point of presence (PoP). Once inside Google's network, traffic is efficiently routed within the network, minimizing hops and latency between your clusters.

Proprietary + Confidential

NEGs send traffic to dynamically configured endpoints



Endpoints include:

- Serverless offerings like Cloud Run Functions or Cloud Run.
- Primary IPs like Compute Engine VMs.
- Secondary IPs like pods on Kubernetes for container-native load balancing.

Google Cloud

Say: To route the traffic, the global HTTPs load balancer uses NEGs to send traffic to dynamically configured endpoints, including:

- Serverless offerings like Cloud Run Functions or Cloud Run.
- Primary IPs like Compute Engine VMs.
- Secondary IPs like pods on Kubernetes for container-native load balancing.

Proprietary + Confidential

Fleet Networking

01 Fleet networking communications

02 Pod discovery in GKE

03 Multi-cluster Services

04 Configuring multi-cluster Services

05 Multi-cluster gateway

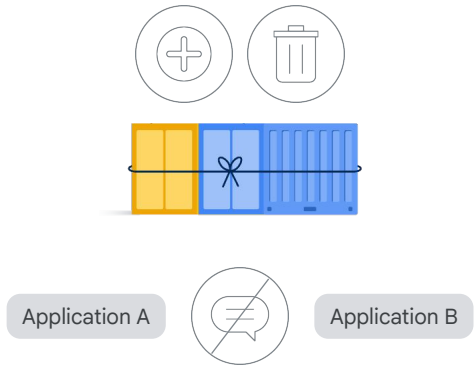
06 Configuring multi-cluster gateways

Google Cloud

Say: So how do Pods in a Kubernetes cluster find and communicate with each other? In this section, you explore the Pod discovery process.

Proprietary + Confidential

Kubernetes provides service discovery mechanisms

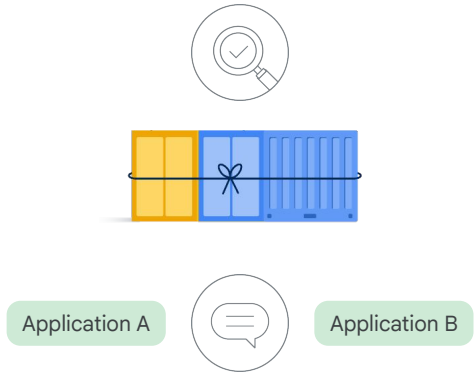


Google Cloud

Say: Kubernetes Pods are ephemeral, meaning they can be created and destroyed dynamically. Because they are frequently changing, it can be difficult for applications to locate and communicate with each other.

Proprietary + Confidential

Kubernetes provides service discovery mechanisms

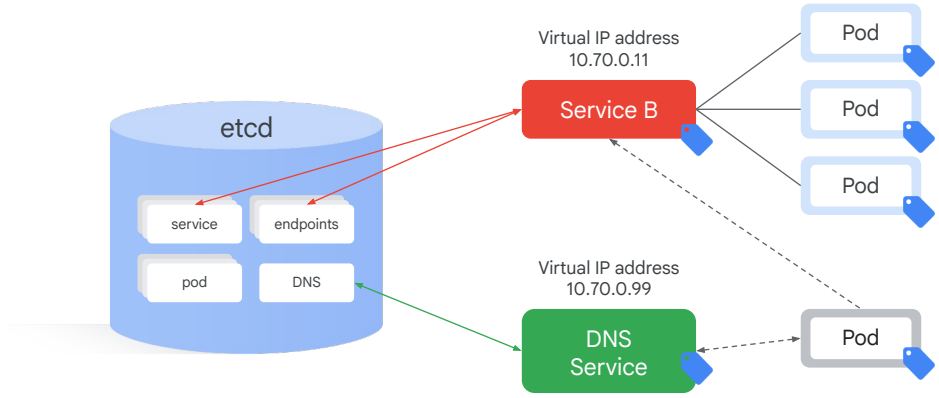


Google Cloud

Say: To address this challenge, Kubernetes provides service discovery mechanisms.

Proprietary + Confidential

Kubernetes service discovery



Google Cloud

Say: For example, let's say that a service named "**Service B**" is defined with a service object in **etcd**, the Kubernetes key-value store.

The Service B definition includes a selector that's used to identify pods that belong to the service.

Kubernetes creates an **Endpoints** object that contains the IP addresses of the pods matching the selector.

The `kube-proxy` component, running on each node, uses the Endpoints information to update `iptables` (or `ipvs`) rules. These rules direct traffic for the Service to its underlying pods.

Most Kubernetes clusters have a DNS server (like CoreDNS) running in the control plane.

The DNS server maps service names to their VIPs. When a Pod needs to communicate with Service B, it performs a DNS lookup on the service name (for example `serviceb.default.svc.cluster.local`).

Once the pod receives the VIP for Service B, it sends traffic to that IP address. `kube-proxy` then intercepts this traffic and redirects it to one of the pod IPs based on the `iptables` rules. Service discovery and request routing are handled efficiently within the cluster.

Proprietary + Confidential

How can you **enable service discovery** for a service spanning **multiple clusters**?

Google Cloud

Say: Traditional Kubernetes service discovery mechanisms are limited to a single cluster. How can you enable service discovery for a service spanning multiple clusters?

Proprietary + Confidential

Cloud Service Mesh

- Enables discoverability and communication between pods in different clusters.
- Provides a unified service mesh across clusters.
- Manage traffic, security, and observability in a multi-cluster environment.
- Multi-cluster Services uses CSM to keep track of endpoints and their health across clusters.

Google Cloud

Say: This is where **Cloud Service Mesh** comes in, enabling discoverability and communication between pods in different clusters. It provides a unified service mesh across your clusters, allowing you to manage traffic, security, and observability for your applications in a multi-cluster environment.

Multi-cluster Services uses Cloud Service Mesh as a control plane to keep track of endpoints and their health across clusters.

Proprietary + Confidential

Fleet Networking

01 Fleet networking communications

02 Pod discovery in GKE

03 Multi-cluster Services

04 Configuring multi-cluster Services

05 Multi-cluster gateway

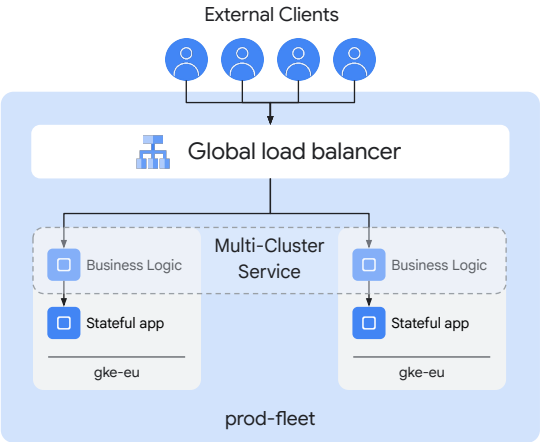
06 Configuring multi-cluster gateways

Google Cloud

Say: GKE offers multi-cluster Services (MCS) for easily running a Service across multiple clusters.

Proprietary + Confidential

MCS uses Cloud DNS to enable service discovery



Multi-cluster Services are made discoverable and accessible across clusters via a virtual IP.

Cloud DNS automatically manages DNS zones and records for exported services.

Multi-cluster Service is only available for GKE clusters, and those clusters must be using VPC-native networking.

Google Cloud

Say: Multi-cluster Services are made externally discoverable and accessible across clusters via a virtual IP.

MCS uses Cloud DNS to enable cross-cluster service discovery. It automatically manages DNS zones and records for exported services, allowing clusters to connect seamlessly.

Multi-cluster Service configuration is separate from Istio and Cloud Service Mesh.

Multi-Cluster Service is only available for GKE clusters, and those clusters must be using VPC-native networking.

Proprietary + Confidential

Fleet Networking

01 Fleet networking communications

02 Pod discovery in GKE

03 Multi-cluster Services

04 Configuring multi-cluster Services

05 Multi-cluster gateway

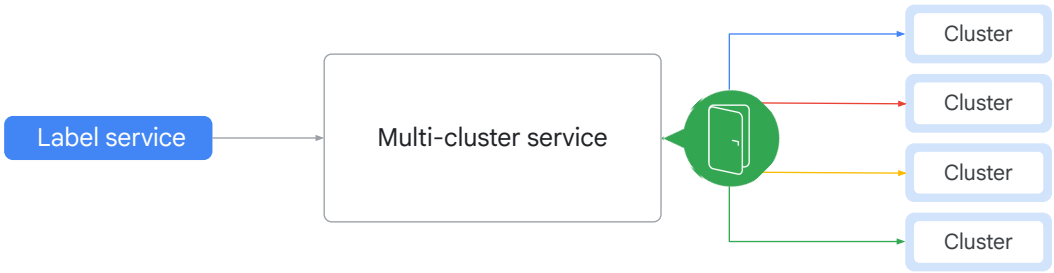
06 Configuring multi-cluster gateways

Google Cloud

Say: While a multi-cluster Service shares similarities with regular Kubernetes Services, it operates differently.

Proprietary + Confidential

MCS acts as a central definition



Google Cloud

Say: Instead of directly routing traffic, an MCS acts as a central definition for a service that spans multiple clusters. This allows the multi-cluster Gateway to manage traffic across these clusters using a single, unified resource.

Proprietary + Confidential

An MCS selects pods using labels and clusters

```
kind: MultiClusterService
apiVersion:
gateway.networking.k8s.io/v1beta1
metadata:
  name: foo
spec:
  template:
    spec:
      selector:
        app: foo
      ports:
      - name: web
        port: 80
```

```
kind: Service
metadata:
  annotations:
    cloud.google.com/neg: '{{..}}'
    cloud.google.com/neg-status: '{{..}}'
    networking.gke.io/mc-parent: '{{..}}'
  name: foo
spec:
...
```

MCS selects pods using labels and clusters.

MCS generates derived Services in target clusters.

The derived Service creates a NEG that tracks POD endpoints.

Google Cloud

Say: A MCS selects pods using labels and clusters. When selecting a cluster, MCS chooses from clusters registered to the fleet, called member clusters.

After MCS has selected a target cluster, it generates a derived Service. Then the derived Service creates an NEG, unless cluster selectors are implemented. The NEG tracks pod endpoints for all pods that match the specified label selector in the cluster. If there are no matching pods in the target cluster, the Service and NEG will be empty.

In this example, the MCS deploys a derived Service in all member clusters with the selector app: foo. If app: foo pods exist in that cluster, then those pod IPs will be added as backends for the multi-cluster Gateway. Note that the MCS is acting as the blueprint for the matching clusters to create a Derived Service from.

Proprietary + Confidential

Enable MCS

1. **Enable the required APIs.**

   MCS, Fleet Hub, Resource Manager, Cloud Service Mesh, Cloud DNS

2. **Enable the MCS feature.**

```
gcloud container fleet \
multi-cluster-services enable \
--project PROJECT_ID
```

3. **Register clusters to a fleet.**

```
gcloud container fleet memberships \
register MEMBERSHIP_NAME \
--gke-cluster \
CLUSTER_LOCATION/CLUSTER_NAME \
--enable-workload-identity \
--project PROJECT_ID
```

Google Cloud

Say: To enable MCS, the first step is to enable the required APIs: MCS, Fleet Hub, Resource Manager, Cloud Service Mesh, and Cloud DNS.

Next, use the `gcloud container fleet multi-cluster-services enable` command to enable the MCS feature for your project's fleet.

Then use the `gcloud container fleet memberships register` command to register your GKE clusters to the fleet. It's recommended that you use Workload Identity Federation for GKE.

Proprietary + Confidential

Enable MCS

4 Grant IAM permissions for MCS Importer.

```
gcloud projects \
add-iam-policy-binding PROJECT_ID \
--member \
"serviceAccount:PROJECT_ID.svc.id.goo
g[gke-mcs/gke-mcs-importer]" \
--role "roles/compute.networkViewer"
```

5 Ensure that each cluster in the fleet has a namespace to share Services in.

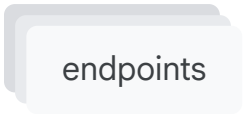
Google Cloud

Say: Use the gcloud projects add-iam-policy-binding command to grant the required Identity and Access Management (IAM) permissions for MCS Importer.

Finally, ensure that each cluster in the fleet has a namespace to share Services in.

Proprietary + Confidential

Derived services



Derived Services

Logical groupings of endpoints that are fully-managed by MCS.

- Used to manage the NEG lifecycle.
- Can serve as backends for multi-cluster gateways.
- Created as headless services.

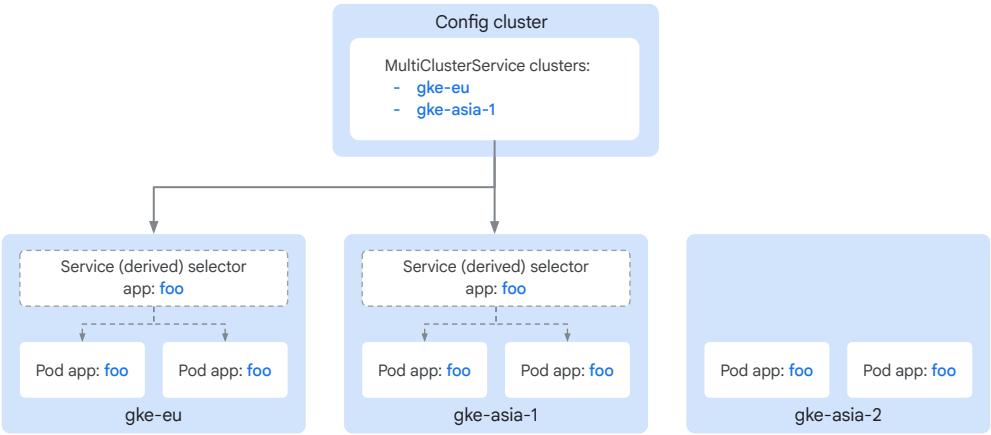
Google Cloud

Say: Let's explore derived Services. Derived Services are logical groupings of endpoints, and they're fully-managed by MCS. They're used to manage the NEG lifecycle, and can serve as backends for multi-cluster Gateways, a cross-cluster ingress resource.

Derived Services are created as headless Services. The MCS controller manages their lifecycle, using only the Selector and Ports fields from the original MCS definition.

Proprietary + Confidential

You can explicitly select clusters for deployment



Google Cloud

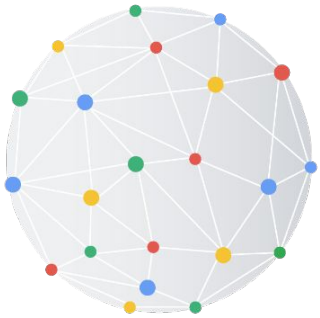
Say: While MCS typically schedules derived services on all target clusters, you can explicitly select specific clusters for deployment. This allows you to control where your service instances run.

There are many use cases where you may want to apply ingress rules to specific clusters:

- Isolating the config cluster to prevent MCSs from selecting across them.
- Controlling traffic between clusters in a blue-green fashion for app migration.
- Routing to application backends that only exist in a subset of clusters.
- Using a single L7 VIP for host/path routing to backends that live on different clusters.

Proprietary + Confidential

Ingress rules can be applied to clusters



- Isolate the config cluster to prevent MCSs from selecting across them.
- Control traffic between clusters during app migration by using a blue/green strategy.
- Route to application backends that only exist in specific clusters.
- Use a single L7 VIP for host/path routing to backends that live on different clusters.

Google Cloud

Say: Ingress rules can be applied to specific clusters, to:

- Isolate the config cluster to prevent MCSs from selecting across them.
- Control traffic between clusters during app migration by using a blue/green strategy.
- Route to application backends that only exist in specific clusters.
- Use a single L7 VIP for host/path routing to backends that live on different clusters.

Proprietary + Confidential

Use the clusters field to select clusters

```
kind: MultiClusterService
apiVersion:
gateway.networking.k8s.io/v1beta1
metadata:
  name: foo
...
clusters:
  - link: “europe-west1-c/gke-eu”
  - link: “asia-northwest1-a/gke-asia-1”
```

- Ensure unique cluster names within the same region and fleet to avoid conflicts.
- This configuration is useful for onboarding or migrating to new clusters and controlling traffic independently of pod deployment.

Google Cloud

Say: To select clusters for your MCS, use the clusters field, specifying each cluster as <region | zone>/<name>. Ensure unique cluster names within the same region and fleet to avoid conflicts.

In this example, the MCS “foo” has a clusters field that references europe-west1-c/gke-eu and asia-northeast1-a/gke-asia.

Pods with matching labels in the gke-asia and gke-eu clusters can be included as backends for a given multi-cluster Gateway. This will exclude the gke-us cluster from ingress even if it has pods with the app: foo label.

This configuration is useful for onboarding or migrating to new clusters and controlling traffic independently of pod deployment.

Proprietary + Confidential

Fleet Networking

01 Fleet networking communications

02 Pod discovery in GKE

03 Multi-cluster Services

04 Configuring multi-cluster Services

05 Multi-cluster gateway

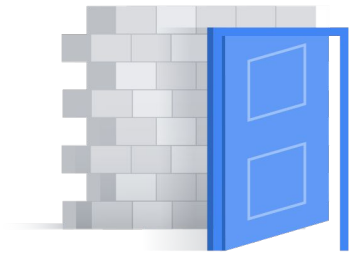
06 Configuring multi-cluster gateways

Google Cloud

Say: Next, let's explore multi-cluster Gateway, a component used to control network ingress from outside a cluster.

Proprietary + Confidential

Multi-cluster gateway



Multi-cluster gateway

- Provisions Google Cloud HTTPs Load Balancers to route traffic across clusters and regions.
- Ensures simple, secure, and scalable management of application networking across clusters.
- Routing capabilities include splitting, traffic mirroring, and health-based failover.

Google Cloud

Say: Multi-cluster Gateway, or MCG, is a cloud-hosted controller that provisions Google Cloud HTTPs load balancers to route traffic across clusters and regions.

It's the successor to Multi-Cluster Ingress (MCI), providing more advanced features and aligning with the Kubernetes Gateway API standard.

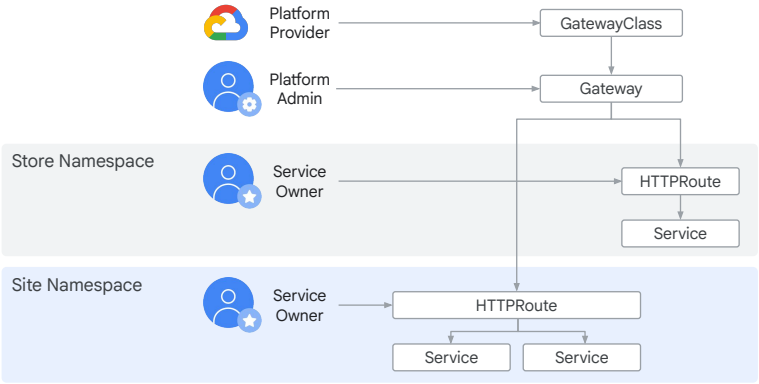
Multi-cluster Gateway ensures simple, secure, and scalable management of application networking across clusters. Examples of its routing capabilities include traffic splitting, traffic mirroring, and health-based failover.

.

Proprietary + Confidential

Multi-cluster gateway is based on the Kubernetes gateway API

Designed to be role-oriented, portable, expressive, and extensible.



Google Cloud

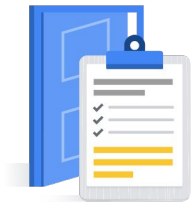
Say: Multi-cluster Gateway is based on the Kubernetes Gateway API, and it's designed to be role-oriented, portable, expressive, and extensible.

The Google Gateway API is the GKE Gateway controller, and it's available for single or multi-clusters. This Google-hosted controller is used to manage the underlying Cloud Load Balancing resources.

The Gateway API has several key resources, including GatewayClass, Gateways, routes, and policies. Let's examine these resources in detail.

Proprietary + Confidential

GatewayClass key resources

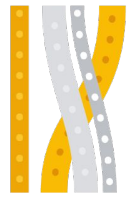


Gateway class

Provides a template for creating load balancers.

Gateway

Define where and how load balancers listen for traffic.



HTTPRoutes

Define rules for routing HTTP(S) requests from a Gateway to a service.

Policies

Define implementation-specific characteristics of a Gateway resource.

Google Cloud

Say: **GatewayClass** provides a template for creating load balancers. It defines the type of load balancer and its capabilities. In GKE, different GatewayClasses correspond to different Google Cloud load balancers.

Gateways define where and how load balancers listen for traffic. Cluster operators create Gateways based on a GatewayClass.

HTTPRoutes define rules for routing HTTP(S) requests from a Gateway to Kubernetes services.

Policies define implementation-specific characteristics of a Gateway resource (for example, health checks and frontend or backend configurations). Policies are attached to Gateways to configure specific behaviors.

Proprietary + Confidential

Fleet Networking

01 Fleet networking communications

02 Pod discovery in GKE

03 Multi-cluster Services

04 Configuring multi-cluster Services

05 Multi-cluster gateway

06 Configuring multi-cluster gateways

Google Cloud

Say: When configuring a multi-cluster Gateway, the first step is to to configure the Google Cloud load balancer in the Gateway custom resource definition (CRD).

Proprietary + Confidential

Gateway YAML file

The **gatewayClassName** field specifies the controller and determines the type of load balancer that will be provisioned.

```
kind: Gateway
apiVersion: gateway.networking.k8s.io/v1beta1
metadata:
  name: external-http
  namespace: store
spec:
  gatewayClassName: gke-l7-gxlb-mc
  listeners:
  - protocol: HTTPS
    port: 443
    routes:
      kind: HTTPRoute
      selector:
        matchLabels:
          gateway: external-http
    tls:
      mode: Terminate
      options:
        .../pre-shared-certs: cert
```

Google Cloud

Say:
Our first step is to ensure that the Gateway API is enabled. The Gateway API is usually enabled by default on newer GKE clusters. However, ensure that your GKE version supports it

In the Gateway YAML file, specify the name of the resource and the target namespace.

In the **`spec`** section, use the **`gatewayClassName`** field to specify which controller you want to use. The **`gatewayClassName`** determines the type of load balancer that will be provisioned.

Proprietary + Confidential

Load balancer types

- gke-l7-global-external-managed
- gke-l7-regional-external-managed
- gke-l7-gxlb
- gke-l7-rilb
- gke-l7-gxlb-mc
- gke-l7-rilb-mc
- gke-l7-global-external-managed-mc
- gke-l7-regional-external-managed-mc

```
kind: Gateway
apiVersion: gateway.networking.k8s.io/v1beta1
metadata:
  name: external-http
  namespace: store
spec:
  gatewayClassName: gke-l7-gxlb-mc
  listeners:
  - protocol: HTTPS
    port: 443
    routes:
      kind: HTTPRoute
      selector:
        matchLabels:
          gateway: external-http
    tls:
      mode: Terminate
      options:
        .../pre-shared-certs: cert
```

Google Cloud

Say: Examples of load balancer types include:

- **gke-l7-global-external-managed:** Global external HTTP(S) managed load balancer.
- **gke-l7-regional-external-managed:** Regional external HTTP(S) managed load balancer.
- **`gke-l7-gxlb`:** Global external HTTP(S) load balancer.
- **`gke-l7-rilb`:** Regional internal HTTP(S) load balancer.
- **`gke-l7-gxlb-mc`:** Multi-cluster global external HTTP(S) load balancer.
- **`gke-l7-rilb-mc`:** Multi-cluster regional internal HTTP(S) load balancer.
- **gke-l7-global-external-managed-mc:** Multi-cluster global external HTTP(S) managed load balancer.
- **gke-l7-regional-external-managed-mc:** Multi-cluster regional external HTTP(S) managed load balancer.

To load balance across clusters, include **`-mc`** at the end of the load balancer name.

Proprietary + Confidential

Routes

If the route is located within a separate HTTPRoute, use a selector to link it with the Gateway.

For HTTPS load balancers, specify the credentials for TLS encryption.

```
kind: Gateway
apiVersion: gateway.networking.k8s.io/v1beta1
metadata:
  name: external-http
  namespace: store
spec:
  gatewayClassName: gke-l7-gxlb-mc
  listeners:
  - protocol: HTTPS
    port: 443
    routes:
      kind: HTTPRoute
      selector:
        matchLabels:
          gateway: external-http
    tls:
      mode: Terminate
      options:
        .../pre-shared-certs: cert
```

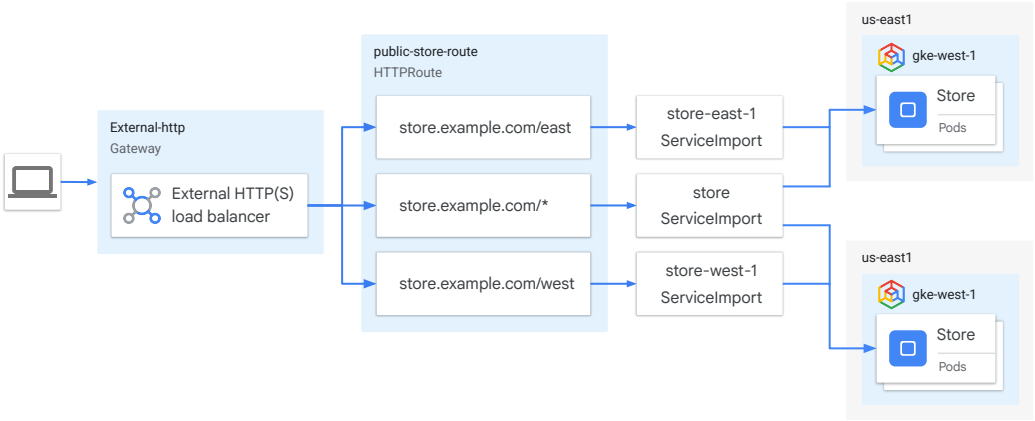
Google Cloud

Say: You can create routes inside the Gateway or in a separate **HTTPRoute** resource. If the route is located within a separate HTTPRoute, use a selector to link it with the Gateway.

For HTTPS load balancers, specify the credentials for TLS encryption.

Proprietary + Confidential

ServiceImports



Google Cloud

Say: In a multi-cluster gateway, HTTPRoutes use ServiceImports to identify services residing in other clusters. This abstraction is necessary because, unlike single-cluster gateways, a multi-cluster gateway needs a way to reference services that may not exist locally.

Proprietary + Confidential

An HTTPRoute routes requests to backend services

```yaml
kind: HTTPRoute
apiVersion: gateway.networking.k8s.io/v1beta1
metadata:
  name: store-route
  namespace: store
  labels:
    gateway: external-http
  rules:
  - forwardTo: external-http
    - backendRef:
        group: net.gke.io
        kind: ServiceImport
        name: store
- matches:
    - path:
        type: Prefix
        value: /west
[…]
    backendRefs:
      - group: net.gke.io
        kind: ServiceImport
        name: store-west-1
      port: 8080
  - matches:
    - path:
        type: Prefix
        value: /east
    backendRefs:
      - group: net.gke.io
        kind: ServiceImport
        name: store-east-1
      port: 8080
```

Google Cloud

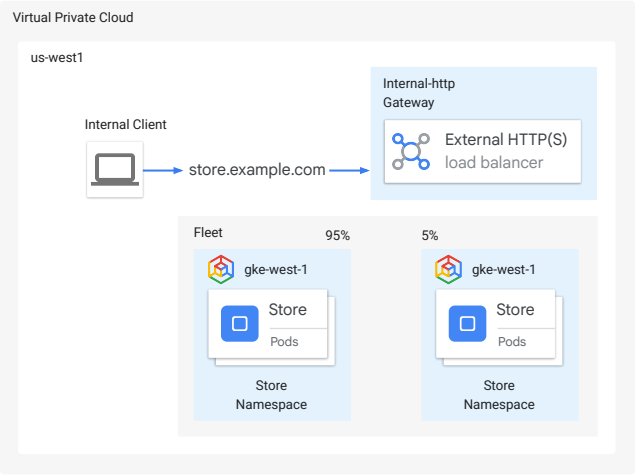
Say: An HTTPRoute defines rules for routing HTTP(S) requests from a Gateway to backend services.

In the YAML file, the connection with the Gateway is defined, and “store” is configured as the default route.

Paths to /east and /west are configured to route to multi-cluster Services called store-west-1 and store-east-1.

Proprietary + Confidential

HTTPRoutes are flexible



Traffic splits are useful for:

- Canary deployments
- Blue-green deployments
- A/B Testing

Google Cloud

Say: HTTPRoutes are flexible and use features like traffic splits to deliver requests based on weights to services in the same or different clusters.

Traffic splits are useful for:

- **Canary deployments**, where you gradually move traffic to the new version of your application to identify issues before they affect all instances of the application.
- **Blue-green deployments**, where you create a completely new Deployment with a newer version of the application. When the Pods are ready, the traffic can be switched from the old blue version to the new green version. Then, Pods in the blue deployment version will be deleted.
- **A/B testing**, where you deploy multiple versions of your application, divert the same percentage of traffic to each version, and test conversion rates to see which version is performing better.

Proprietary + Confidential

Use the weight field to split traffic

```
kind: HTTPRoute
apiVersion: gateway.networking.k8s.io/v1beta1
metadata:
  name: store-route
  namespace: store
  labels:
    gateway: internal-http
  rules:
   backendRefs:
      - group: net.gke.io
        kind: ServiceImport
        name: store-west-1
      port: 8080
      weight: 90
```

```
[…]

 backendRefs:
      - group: net.gke.io
        kind: ServiceImport
        name: store-east-1
      port: 8080
      weight: 10
```

Google Cloud

Say: To split traffic, you can use the weight field in the backendRefs of an HTTPRoute. If no weight is specified, traffic is distributed evenly across all backends using a round-robin strategy.

Proprietary + Confidential

Use BackendConfig to enable additional features

```yaml
apiVersion: networking.gke.io/v1
kind: GCPBackendPolicy
metadata:
  name: my-backend-policy
  namespace: lb-service-namespace
spec:
  default:
    connectionDraining:
      drainingTimeoutSec: 60
  targetRef:
    group: net.gke.io
    kind: ServiceImport
    name: lb-service
```

Google Cloud

Say: The GatewayClass of a multi-cluster Gateway supports the GCPBackendPolicy resource, where you can customize backend settings on a per-Service level. This enables you to benefit from additional Google Cloud features.

Proprietary + Confidential

Additional Google Cloud features

```yaml
apiVersion: networking.gke.io/v1
kind: GCPBackendPolicy
metadata:
  name: my-backend-policy
  namespace: lb-service-namespace
spec:
  default:
    connectionDraining:
      drainingTimeoutSec: 60
  targetRef:
    group: net.gke.io
    kind: ServiceImport
    name: lb-service
```

ConnectionDraining: completes connections when a backend is removed

Google Cloud

Say: You can use **ConnectionDraining** to enable existing connections to complete when a backend is removed. By default the value is 0.

Proprietary + Confidential

Additional Google Cloud features

```
apiVersion: networking.gke.io/v1
kind: GCPBackendPolicy
metadata:
  name: backend-policy
spec:
  default:
    iap:
      enabled: true
      oauth2ClientSecret:
        name: SECRET_NAME
      clientID: CLIENT_ID
  targetRef:
    group: net.gke.io
    kind: ServiceImport
    name: lb-service
```

IAP: Authenticate and authorize employees to use internal applications

Google Cloud

Say:

IAP enables you to authenticate and authorize employees to use internal applications.

Proprietary + Confidential

Additional Google Cloud features

```
apiVersion: networking.gke.io/v1   […]
kind: GCPBackendPolicy
metadata:
  name: my-backend-policy
  namespace: lb-service-namespace
spec:
  default:
    securityPolicy: example-policy
  targetRef:
    group: net.gke.io
    kind: ServiceImport
    name: lb-service
```

```
logging:
  enabled: true
  sampleRate: 500000
  optionalMode: CUSTOM
  optionalFields:
  - tls.cipher
  - orca_load_report.cpu_utilization
```

Security Policies: protect applications from web-based attacks

Logging: Authenticate and authorize employees to use internal applications

Google Cloud

Say:

Cloud Armor Security Policies help you protect your load-balanced applications from web-based attacks such as denial of service, cross-scripting injection, or SQL injection. Enabling Logging can log all HTTP requests from clients to Cloud Logging at a sampling rate of your choice.